Exactness of Marginal Inference in Logical Credal Networks

A consolidated reference: precise statements, complete proofs, and a verified example for every result

IBM Research

Abstract

This document is the single reference for *when* the LCN library's marginal-inference engines return exact bounds, and for *what* they bound when they do not. It fixes one framework — the LCN distribution set $\mathcal{D}_{\mathrm{LCN}}$, the strong extension $\mathcal{D}_{\mathrm{SE}} \supseteq \mathcal{D}_{\mathrm{LCN}}$ of the compiled credal network, and a bound-character taxonomy that always names its reference set — and then places each engine in it. Every proposition and theorem carries a *complete* proof rather than a sketch, and every technical result is accompanied by a concrete example whose numbers were produced by running the shipped engines.

The central definition is *family-realizability* (Definition 2), stated here for general chain-graph LCNs, in which a family is a whole undirected chain component conditioned on its parents. Around it we prove: the inclusion $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{D}_{\mathrm{SE}}$ (Proposition 1); a sufficient condition for equality (Proposition 2) that, notably, does *not* require single-connectedness; the exactness of Credal VE over $\mathcal{D}_{\mathrm{SE}}$ for unconditional marginals (Theorem 1); Interval BP's outer guarantee and its exactness on trees (Theorem 2); ApproxLP's inner guarantee over $\mathcal{D}_{\mathrm{SE}}$ (Theorem 3); the CredalJT exactness theorem at treewidth cost under hypotheses H1–H4 (Theorem 4); and, for reference, ARIEL's message-soundness lemma together with the precise form of its exactness claim (Section 6).

Three corrections to the companion notes are folded in, each established by measurement rather than argument. Credal CTE is *not* exact at ϵ=0: on `d4_biting` it returns $P(B) = [0.05, 0.65]$ strictly inside the exact $[0.04, 0.72]$, so it is not even a valid outer bound. The characterization "$\mathcal{D}_{\mathrm{SE}}$=$\mathcal{D}_{\mathrm{LCN}}$ iff singly connected and fully realizable" cannot be an *iff* over every atom — `d4_biting` has $\mathcal{D}_{\mathrm{SE}} \supsetneq \mathcal{D}_{\mathrm{LCN}}$ yet exact marginals. And single-connectedness is not what governs Gap B: on a *loopy* realizable k-tree, Credal VE is exact on every atom. We also record where the machinery has genuine holes — dominance pruning interacts badly with ratio read-off, the conditional-query denominator floors are soundness hypotheses, and `ExactInference`-global fails to certify several atoms of a five-atom instance — and we state these as caveats instead of concealing them.

Contents

1 The framework

1.1 LCNs and the LCN distribution set

An LCN over binary atoms $\mathbf{V} = \{V_1, \ldots, V_n\}$ is a finite set of probability *sentences*, each of Type-1, $\ell \le P(\varphi) \le u$, or Type-2, $\ell \le P(\varphi \mid \psi) \le u$, over propositional formulas, together with the conditional independencies of the *Local Markov Condition* (LMC) of its primal graph. A candidate joint $p \in \Delta(2^{\mathbf{V}})$ over the 2^n truth assignments is a member of the *LCN distribution set* $\mathcal{D}_{\mathrm{LCN}}$ when it satisfies every sentence and every LMC equality.

Type-2 sentences: the cleared form. A Type-2 sentence is a ratio constraint, and a ratio is undefined when its conditioning event has probability zero. Every engine in the library therefore implements the algebraically *cleared* form

$$\ell \cdot P(\psi) \;\le\; P(\varphi \wedge \psi) \;\le\; u \cdot P(\psi), \tag{1}$$

which is total (defined for all p) and *bilinear* in p. We define $\mathcal{D}_{\mathrm{LCN}}$ with (1), not with the ratio, because that is the set the engines actually target.

Remark 1 (The two readings differ)**.** On $\{p : P(\psi) = 0\}$ the cleared form (1) is satisfied vacuously, whereas the ratio $P(\varphi \mid \psi)$ is undefined. Consequently the cleared reading admits joints the ratio reading excludes. Defining $\mathcal{D}_{\mathrm{LCN}}$ by the ratio — as earlier versions of this note did — would compare every engine against a set that none of them optimizes over. All results below are stated for the cleared reading; they transfer to the ratio reading on the subset where every conditioning event has positive probability.

LMC equalities. Each LMC assertion $(X \perp\!\!\!\perp Y \mid S)$ imposes, for *every* joint configuration $\mathbf{x}, \mathbf{y}, \mathbf{s}$ of the atom blocks X, Y, S, the bilinear equality

$$P(\mathbf{x}, \mathbf{y}, \mathbf{s})\, P(\mathbf{s}) = P(\mathbf{x}, \mathbf{s})\, P(\mathbf{y}, \mathbf{s}), \tag{2}$$

i.e. $X \perp\!\!\!\perp Y \mid S$ under p. These are built by `lmc_constraint_groups_vec` (`lcn/inference/utils/common.py`). Writing g_s for the functional a sentence s constrains and $h_\iota(p) = 0$ for the equalities of assertion ι,

$$\mathcal{D}_{\mathrm{LCN}} = \left\{ p \in \Delta(2^{\mathbf{V}}) \;:\; \ell_s \le g_s(p) \le u_s \;\forall s, \qquad h_\iota(p) = 0 \;\forall \iota \right\}, \tag{3}$$

where a Type-1 $g_s(p) = P(\varphi) = \mathbf{1}_\varphi^\top p$ is linear and a Type-2 sentence contributes the two rows of (1). The marginal task for a query atom a is

$$\underline{P}(a) = \min_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a), \qquad \overline{P}(a) = \max_{p \in \mathcal{D}_{\mathrm{LCN}}} P(a),$$

and analogously for a conditional $P(a \mid e)$ under evidence e.

1.2 Locality

Write $p \restriction A$ for the marginal of p on an atom set A. The following lemma is used by every proof in this document; in the companion notes it appears as an unproved remark.

Lemma 1 (Locality)**.** *Every constraint in* (3) *depends on p only through its marginal on a fixed atom set, its* scope*: a sentence s through $p \restriction \mathrm{atoms}(s)$, and an LMC assertion $(X \perp\!\!\!\perp Y \mid S)$ through $p \restriction (X \cup Y \cup S)$. Consequently, if p and p' satisfy $p \restriction A = p' \restriction A$ for the scope A of a constraint c, then c holds for p iff it holds for p'.*

Proof. Let c be a constraint with scope A. There are three cases.

Type-1. $P(\varphi) = \sum_{\omega \models \varphi} p(\omega)$. Since $\mathrm{atoms}(\varphi) \subseteq A$, the truth value of φ at ω is determined by $\omega \restriction A$; grouping the sum by the value of $\omega \restriction A$ gives $P(\varphi) = \sum_{\alpha \models \varphi} (p \restriction A)(\alpha)$, where α ranges over assignments of A. This is a function of $p \restriction A$ alone.

Type-2. By the previous case both $P(\varphi \wedge \psi)$ and $P(\psi)$ are functions of $p \restriction A$ with $A \supseteq \mathrm{atoms}(\varphi) \cup \mathrm{atoms}(\psi)$; the rows (1) are therefore functions of $p \restriction A$.

LMC. Each factor in (2) is a probability of a conjunction of literals over $X \cup Y \cup S$, hence by the Type-1 case a function of $p \restriction (X \cup Y \cup S)$; a product of such functions is again one.

In each case the constraint's truth value is determined by $p \restriction A$, which is precisely the stated consequence. □

1.3 Chain graphs, families, and the credal network

The LCN's structure graph is simplified by collapsing each connected component of its undirected part into a meta-node (`LCN.simplify_structure_graph`). When the result is a directed acyclic graph the LCN is a *chain graph*, and the joint factorizes over *families*

$$P(\mathbf{V}) = \prod_{v} P(\mathrm{ch}_v \mid \mathrm{pa}_v), \tag{4}$$

one per node v of the simplified graph, where ch_v is the (possibly multi-atom) chain component at v and pa_v its parents. The family's *scope* is $\mathrm{ch}_v \cup \mathrm{pa}_v$. Root families have $\mathrm{pa}_v = \varnothing$, so their factor is the marginal $P(\mathrm{ch}_v)$.

Example 1 (ch_v is genuinely a block)**.** Multi-atom chain components are the rule, not an edge case. Running `LCN.process_chain_graph`:

- `examples/alarm.lcn` yields families $A \mid B, E$; B; E; and $\boxed{C\text{-}D \mid A}$ — a two-atom component $\mathrm{ch}_v = \{C, D\}$, because `s4` couples C and D undirectedly.
- `examples/smokers.lcn` yields $\boxed{F_1F_2F_3 \mid \varnothing}$ and $\boxed{S_1S_2S_3 \mid F_1F_2F_3}$ — two *three*-atom components — besides $C_i \mid S_1S_2S_3$.

Any definition of realizability phrased for a single child atom cannot even be stated on these instances; Definition 2 is therefore given for blocks.

Local credal sets and the strong extension. For each family the `cn/` pipeline (`local_credal_sets.py` with `method="linear"`) computes a *local credal set* K_v: the set of conditional tables $P(\mathrm{ch}_v \mid \mathrm{pa}_v)$ attainable by *some* joint satisfying the sentences whose atom scope lies inside the family scope. Operationally it is a linear-fractional program $P(\mathrm{ch}_v, \mathrm{pa}_v)/P(\mathrm{pa}_v)$ solved by Charnes–Cooper, in which *the parents' distribution* $P(\mathrm{pa}_v)$ *is a free variable.* The credal-network engines bound the *strong extension*

$$\mathcal{D}_{\mathrm{SE}} = \Big\{ \textstyle\prod_v P_v \;:\; P_v \in K_{v,\pi} \text{ chosen independently for each parent configuration } \pi \Big\}. \tag{5}$$

Remark 2 (Row-wise versus table-wise, a third looseness source)**.** Equation (5) is the *row-independent* product $\prod_v \prod_\pi K_{v,\pi}$: the code (`cve.py`, via `itertools.product` over parent configurations) chooses one conditional row per parent configuration independently. This is in general *strictly larger* than {tables lying in K_v}, since K_v need not be a Cartesian product of its rows. The companion notes use both readings interchangeably; we fix the row-wise reading of (5) throughout, because it is the one implemented. Note that this is a source of looseness distinct from Gap A and Gap B below, and unnamed by that taxonomy.

Remark 3 (The vertex machinery needs `method="linear"`)**.** Under `method="linear"` each $K_{v,\pi}$ is a polytope (the Charnes–Cooper image of a polyhedron), so it has finitely many extreme points and $\mathcal{D}_{\mathrm{SE}}$ is a product of polytopes — the hypothesis every vertex-enumeration argument below needs. Under scheme D1 (`method="linear-tight"`) the local program is additionally cut by *bilinear* LMC equalities, so $K_{v,\pi}$ need not be convex; vertex enumeration then explores a subset of the convex hull, and an engine that is exact over $\mathcal{D}_{\mathrm{SE}}$ for `"linear"` may become *inner* with respect to the D1 strong extension. All exactness results below are stated for `method="linear"`.

1.4 Bound character, relative to a named reference set

Definition 1 (Bound character)**.** Let $\mathcal{R}$ be a reference set (here $\mathcal{D}_{\mathrm{LCN}}$ or $\mathcal{D}_{\mathrm{SE}}$) with true bounds $[\underline{q}^\star, \overline{q}^\star]$ for the query at hand. An engine's interval $[\underline{q}, \overline{q}]$ is *exact w.r.t.* $\mathcal{R}$ if $[\underline{q}, \overline{q}] = [\underline{q}^\star, \overline{q}^\star]$; a valid *outer* bound if $[\underline{q}, \overline{q}] \supseteq [\underline{q}^\star, \overline{q}^\star]$; an *inner* bound if $[\underline{q}, \overline{q}] \subseteq [\underline{q}^\star, \overline{q}^\star]$; and *unreliable* if neither containment is guaranteed.

Naming $\mathcal{R}$ is not pedantry: an engine can be inner w.r.t. $\mathcal{D}_{\mathrm{SE}}$ and simultaneously outer w.r.t. $\mathcal{D}_{\mathrm{LCN}}$, since $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{D}_{\mathrm{SE}}$. ApproxLP is exactly such a case (Example 7), and the unqualified claim "ApproxLP is not a valid outer bound" found in earlier versions of this note is false as stated.

1.5 The two looseness gaps

- **Gap A (local-set widening).** Each K_v is computed per family in isolation, so it may admit conditionals that no global $p \in \mathcal{D}_{\mathrm{LCN}}$ realizes: it ignores sentences on *other* families and the cross-family LMC equalities. Formally $K_v^{\mathrm{true}} \subseteq K_v^{\mathrm{local}}$, sometimes strictly.
- **Gap B (strong-extension widening).** Even with $K_v = K_v^{\mathrm{true}}$, the free product of (5) lets each family pick its row independently, which can violate a sentence whose scope spans several families.

Schemes D1 (`method="linear-tight"`), D2 (`merge_budget`) and D3 (`solver="scip"`) attack Gap A at build time and are free for every credal-network engine; D4 (`coupling="cross-family"`) and D5 (`CredalJT`) attack Gap B at propagation time. See `tighter_approximation.tex` for the taxonomy.

2 Family-realizable sentences

Which sentences can a *single* family's local set enforce? A family's degrees of freedom are the conditional probabilities $P(\mathrm{ch}_v{=}c \mid \mathrm{pa}_v{=}\pi)$; the parents' marginal $P(\mathrm{pa}_v)$ is not one of them — it is determined elsewhere in the network, and is a free variable during the local solve. A sentence that constrains a quantity depending on $P(\mathrm{pa}_v)$ therefore cannot be enforced locally, and the free product (5) may violate it. This is the mechanism behind Gap B, and the following definition isolates it.

Definition 2 (Family-realizable sentence)**.** Let $\mathcal{L}$ be a chain-graph LCN with families $\{(\mathrm{ch}_v, \mathrm{pa}_v)\}$, and let s be a sentence with functional g_s. Say s is *family-realizable in family* v if $\mathrm{atoms}(s) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$ and the value $g_s(p)$ is determined by the conditional table $P(\mathrm{ch}_v \mid \mathrm{pa}_v)$ *alone* — i.e. for any two joints p, p' that induce the same conditional table at v, $g_s(p) = g_s(p')$, whatever the parents' marginals $P(\mathrm{pa}_v)$ and $P'(\mathrm{pa}_v)$ may be. Concretely, s is family-realizable in v if and only if it has one of the two forms:

(R1) *Type-1 on a root block:* s is $\ell \le P(\varphi) \le u$ with $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$ and $\mathrm{pa}_v = \varnothing$. Then $P(\varphi)$ is a function of $P(\mathrm{ch}_v)$, which *is* the family's table.

(R2) *Type-2 pinning one full parent configuration:* s is $\ell \le P(\varphi \mid \psi) \le u$ with $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v$, $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$, and ψ satisfied by *exactly one* assignment $\pi^\star$ of pa_v. Then $P(\varphi \mid \psi) = P(\varphi \mid \mathrm{pa}_v{=}\pi^\star)$ is a single row of the table.

A sentence is *non-realizable* if it is family-realizable in no family. The LCN is *fully realizable* if every sentence is family-realizable in some family.

Note the quantifier: "in *some* family". The library attaches a sentence to every family whose scope contains its atoms (`process_chain_graph`), so one sentence is typically attached to several families while being realizable in only one of them — see Example 2.

The next lemma is what makes Definition 2 a *criterion* rather than a description: it shows the two forms (R1)–(R2) exhaust the $P(\mathrm{pa}_v)$-invariant sentences, so realizability can be decided syntactically.

Lemma 2 (Invariance criterion)**.** *Let v be a family with $\mathrm{pa}_v \neq \varnothing$ and let s be a sentence with $\mathrm{atoms}(s) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$. Write π for assignments of pa_v, $w_\pi = P(\mathrm{pa}_v{=}\pi)$ for the parents' marginal, and $T(\cdot \mid \pi) = P(\mathrm{ch}_v{=}\cdot \mid \mathrm{pa}_v{=}\pi)$ for the table.*

(i) *If s is Type-1, $\ell \le P(\varphi) \le u$, then $P(\varphi) = \sum_\pi w_\pi\, T(\varphi_\pi \mid \pi)$ where $T(\varphi_\pi \mid \pi) := \sum_{c\colon (c,\pi)\models\varphi} T(c \mid \pi)$. This is $P(\mathrm{pa}_v)$-invariant for all tables iff the map $\pi \mapsto T(\varphi_\pi \mid \pi)$ is constant for every table — which fails whenever $|\{\pi\}| \ge 2$ and φ is non-trivial. Hence no Type-1 sentence is realizable in a non-root family.*

(ii) *If s is Type-2 with $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$ and $\Pi_\psi := \{\pi : \pi \models \psi\}$, then on $\{P(\psi) > 0\}$*

$$P(\varphi \mid \psi) = \sum_{\pi\in\Pi_\psi} \lambda_\pi\, T(\varphi_\pi \mid \pi), \qquad \lambda_\pi = \frac{w_\pi}{\sum_{\pi'\in\Pi_\psi} w_{\pi'}}, \tag{6}$$

a convex combination of table rows with weights λ determined by $P(\mathrm{pa}_v)$. If $|\Pi_\psi| = 1$ this is one row, hence invariant; if $|\Pi_\psi| \ge 2$ it is invariant for all tables iff the rows agree, so s is not *realizable.*

Proof. (i) Condition the Type-1 probability on the parent block. Since $\mathrm{atoms}(\varphi) \subseteq \mathrm{ch}_v \cup \mathrm{pa}_v$, the truth value of φ at a joint assignment is determined by $(\mathrm{ch}_v, \mathrm{pa}_v)$, so by Lemma 1 $P(\varphi) = \sum_\pi P(\mathrm{pa}_v{=}\pi)\, P(\varphi \mid \mathrm{pa}_v{=}\pi) = \sum_\pi w_\pi T(\varphi_\pi \mid \pi)$, which is the stated formula. This is an affine function of the weight vector w on the simplex, and an affine function on a simplex is constant iff all its coefficients coincide; hence invariance for *every* table forces $\pi \mapsto T(\varphi_\pi \mid \pi)$ to be constant. Choose a table with $T(\varphi_{\pi_1} \mid \pi_1) = 1$ and $T(\varphi_{\pi_2} \mid \pi_2) = 0$ for two distinct parent configurations $\pi_1 \ne \pi_2$ (possible whenever φ is satisfiable and refutable within ch_v, i.e. non-trivial); then the coefficients differ and invariance fails. So s is not realizable in v.

(ii) With $\mathrm{atoms}(\psi) \subseteq \mathrm{pa}_v$, the event ψ is a union of parent configurations, $\{\psi\} = \bigcup_{\pi\in\Pi_\psi}\{\mathrm{pa}_v{=}\pi\}$, a disjoint union. Hence $P(\psi) = \sum_{\pi\in\Pi_\psi} w_\pi$ and, by the computation in (i) restricted to Π_ψ, $P(\varphi \wedge \psi) = \sum_{\pi\in\Pi_\psi} w_\pi T(\varphi_\pi \mid \pi)$. Dividing gives (6), and λ is a probability vector supported on Π_ψ.

If $|\Pi_\psi| = 1$, say $\Pi_\psi = \{\pi^\star\}$, then $\lambda_{\pi^\star} = 1$ and $P(\varphi \mid \psi) = T(\varphi_{\pi^\star} \mid \pi^\star)$, a single row — invariant, and this is (R2). If $|\Pi_\psi| \ge 2$, then (6) is a non-degenerate convex combination: pick $\pi_1 \ne \pi_2$ in Π_ψ and a table with $T(\varphi_{\pi_1} \mid \pi_1) = 1$, $T(\varphi_{\pi_2} \mid \pi_2) = 0$. Taking w concentrated near π_1 gives $P(\varphi \mid \psi) \to 1$, and near π_2 gives $\to 0$, so the value genuinely depends on $P(\mathrm{pa}_v)$ and s is not realizable. Equivalently, $\partial P(\varphi \mid \psi)/\partial\lambda_\pi = T(\varphi_\pi \mid \pi)$ is not constant across Π_ψ. □

Remark 4 (Atom scope is not the criterion)**.** Lemma 2 shows family-realizability is strictly stronger than the atom-scope test "atoms(s) fits inside a family scope". Three canonical offenders are all atom-scope family-local:

- a bare marginal $P(x)$ on a *non-root* child (Lemma 2(i));
- a child–parent joint $P(x \wedge y)$ with $y = \mathrm{pa}_x$, which equals $P(x \mid y)P(y)$;
- a Type-2 sentence whose ψ is satisfied by two or more parent configurations — e.g. $P(x_2 \mid x_0 \oplus x_1)$ at a collider (Lemma 2(ii)). Note this includes a conjunction over a *strict subset* of the parents, which pins a set of configurations rather than one.

2.1 A five-variable chain-graph example

Example 2 (Family-realizability on a five-atom chain graph)**.** The instance `ktree_fr_k3_n5_1.lcn` from `benchmarks/ktree_small_fr/` (n=5, a family-realizable k-tree with k=3):

$s_0 : 0.01 \le P(x_3) \le 0.399975$
$s_1 : 0.01 \le P(x_4 \mid x_3) \le 0.442867$
$s_2 : 0.638553 \le P(x_2 \mid \neg x_3 \wedge x_4) \le 1.0$
$s_3 : 0.01 \le P(\neg x_1 \mid x_3 \wedge x_4 \wedge x_2) \le 0.307066$
$s_4 : 0.673756 \le P(\neg x_0 \mid x_3 \wedge x_4 \wedge \neg x_1) \le 1.0$
$s_5 : 0.748845 \le P(\neg x_3) \le 0.954666$
$s_6 : 0.141404 \le P(x_3) \le 0.247704$

`LCN.process_chain_graph` reports five singleton-block families and a single LMC assertion:

$$x_3 \mid \varnothing, \quad x_4 \mid x_3, \quad x_2 \mid x_3, x_4, \quad x_1 \mid x_2, x_3, x_4, \quad x_0 \mid x_1, x_3, x_4; \qquad (x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3, x_4).$$

Table 1 classifies every sentence. Two points are worth drawing out. First, the *attached* column (what `process_chain_graph` collects by the scope-subset test) is much larger than the *realizable-in* column: the root marginals s_0, s_5, s_6 are attached to all five families but realizable only in the root family x_3, which is exactly why Definition 2 quantifies existentially over families. Second, each of s_1, s_2, s_3, s_4 conditions on a *full* conjunction of signed literals over all of its family's parents, so $|\Pi_\psi| = 1$ and (R2) applies — this is what the generator's `full_parents=True` flag guarantees for every `-fr` class.

Table 1: Sentence classification for `ktree_fr_k3_n5_1.lcn`. Every sentence is family-realizable in exactly one family, so the instance is fully realizable.

sentence	form	attached to families	realizable in	why
s_0, s_5, s_6	Type-1 $P(x_3)$, $P(\neg x_3)$	all five	$x_3 \mid \varnothing$	(R1) root block
s_1	Type-2, $\psi = x_3$	x_4, x_2, x_1, x_0	$x_4 \mid x_3$	(R2) $\lvert\Pi_\psi\rvert$=1
s_2	Type-2, $\psi = \neg x_3 \wedge x_4$	x_2, x_1	$x_2 \mid x_3, x_4$	(R2) $\lvert\Pi_\psi\rvert$=1
s_3	Type-2, $\psi = x_3 \wedge x_4 \wedge x_2$	x_1	$x_1 \mid x_2, x_3, x_4$	(R2) $\lvert\Pi_\psi\rvert$=1
s_4	Type-2, $\psi = x_3 \wedge x_4 \wedge \neg x_1$	x_0	$x_0 \mid x_1, x_3, x_4$	(R2) $\lvert\Pi_\psi\rvert$=1

Measured bounds. `ExactInference` with `solver="global"` certifies all ten solves (gap 0, status `confirmed`), and `CredalJT` reproduces every endpoint:

engine	$P(x_0)$	$P(x_1)$	$P(x_2)$	$P(x_3)$	$P(x_4)$
`ExactInference` (global)	$[0, 1]$	$[0, 1]$	$[0, 1]$	$[0.141404, 0.247704]$	$[0.001414, 0.921219]$
`CredalJT` (D5, SCIP)	$[0, 1]$	$[0, 1]$	$[0, 1]$	$[0.141404, 0.247704]$	$[0.001414, 0.921219]$

The vacuous $[0, 1]$ on x_0, x_1, x_2 is genuine, not a solver artifact: `cjt_exactness.tex` exhibits explicit feasible witnesses attaining both endpoints (e.g. setting $P(x_2 \mid \neg x_3 \wedge x_4) = 0$ makes s_3's conditioning event unreachable, freeing x_1). The junction tree has maximal cliques $\{x_0, x_1, x_3, x_4\}$ and $\{x_1, x_2, x_3, x_4\}$ — treewidth 3 — and the separator between them is exactly $\{x_1, x_3, x_4\}$, the conditioning set of the lone LMC assertion; this is the certificate that hypotheses H1–H4 of Theorem 4 hold.

Example 3 (The Type-2 half, isolated by a matched pair)**.** To see that clause (R2)'s "exactly one parent configuration" is doing real work — and not merely excluding pathologies — compare two LCNs that differ *only* in the shape of ψ at a collider $x_0, x_1 \to x_2$, with identical structure and identical numeric bounds:

$$\texttt{P0}\colon\ 0.3 \le P(x_0) \le 0.5; \quad \texttt{P1}\colon\ 0.4 \le P(x_1) \le 0.6; \quad \texttt{C}\colon\ 0.2 \le P(x_2 \mid \psi) \le 0.5.$$

ψ	$\lvert\Pi_\psi\rvert$	realizable?	`ExactInference` (global) $P(x_2)$	Credal VE $P(x_2)$
$x_0 \oplus x_1$	2	**no**	$[0.092, 0.770]$	$[\mathbf{0}, \mathbf{1}]$ *vacuous*
$x_0 \wedge x_1$	1	yes	$[0.024, 0.940]$	$[\mathbf{0.024}, \mathbf{0.940}]$ *exact*

With $\psi = x_0 \oplus x_1$ the sentence bounds a $P(\mathrm{pa}_{x_2})$-weighted mixture of two table rows (Lemma 2(ii)); neither row is individually constrained, so every $K_{x_2,\pi}$ is the full box $[0, 1]$ and Credal VE returns the vacuous interval, losing the bound entirely. With $\psi = x_0 \wedge x_1$ the same sentence pins the single row $\pi^\star = (1, 1)$, clause (R2) applies, and Credal VE is exact. Both $P(x_0)$ and $P(x_1)$ are exact ($[0.3, 0.5]$, $[0.4, 0.6]$) in both cases: the damage is local to the offending family. Note also that this is not a solver or enumeration artifact — `strong_extension_exactness.tex` verifies that neither the global SCIP backend nor scheme D1 recovers the bound, because the coupling is not representable as a product of per-configuration credal sets.

Example 4 (The Type-1 half: a marginal on a non-root child)**.** The smallest witness. Take the two-atom chain $x_0 \to x_1$ with

$$\texttt{P0}\colon\ 0.3 \le P(x_0) \le 0.5; \quad \texttt{T0}\colon\ 0.2 \le P(x_1 \mid x_0) \le 0.4; \quad \texttt{T1}\colon\ 0.6 \le P(x_1 \mid \neg x_0) \le 0.8,$$

and optionally add `M1`: $0.45 \le P(x_1) \le 0.55$, a Type-1 marginal on the non-root child x_1 — non-realizable by Lemma 2(i). Measured:

engine	$P(x_1)$ without `M1`	$P(x_1)$ with `M1`
`ExactInference` (global)	$[0.40, 0.68]$	$[\mathbf{0.45}, \mathbf{0.55}]$
`CredalJT` (D5)	$[0.40, 0.68]$	$[\mathbf{0.45}, \mathbf{0.55}]$
Credal VE	$[0.40, 0.68]$	$[0.40, 0.68]$
Interval BP (`interval`)	$[0.40, 0.68]$	$[0.40, 0.68]$
ApproxLP	$[0.40, 0.68]$	$[0.40, 0.68]$

Without `M1` the LCN is fully realizable and all five engines agree. With `M1` the three strong-extension engines are unchanged — the family $P(x_1 \mid x_0)$ cannot enforce a constraint on $P(x_1)$, which depends on $P(x_0)$ — while the two $\mathcal{D}_{\mathrm{LCN}}$ engines tighten to the sentence bound. $P(x_0) = [0.30, 0.50]$ for every engine in both instances.

3 The LCN set versus the strong extension

We now relate $\mathcal{D}_{\mathrm{LCN}}$ and $\mathcal{D}_{\mathrm{SE}}$. Three separate statements are needed, and it is important that they are *not* packaged as a single biconditional — Remark 5 explains why.

Proposition 1 (Inclusion)**.** *Let $\mathcal{L}$ be a chain-graph LCN. Then $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{D}_{\mathrm{SE}}$. Consequently, for every query, $[\underline{q}_{\mathrm{LCN}}, \overline{q}_{\mathrm{LCN}}] \subseteq [\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}]$: any engine that is exact or outer w.r.t. $\mathcal{D}_{\mathrm{SE}}$ is automatically a valid* outer *bound w.r.t. $\mathcal{D}_{\mathrm{LCN}}$.*

Proof. Let $p \in \mathcal{D}_{\mathrm{LCN}}$. Since $\mathcal{L}$ is a chain graph and p satisfies every LMC equality, the chain-graph factorization theorem (Lauritzen–Wermuth–Frydenberg; Lauritzen, *Graphical Models*, Thm. 3.34) gives the factorization (4): $p = \prod_v P_p(\mathrm{ch}_v \mid \mathrm{pa}_v)$, where $P_p(\cdot \mid \cdot)$ denotes the conditionals induced by p (defined arbitrarily on parent configurations of zero probability, which do not affect the product).

Fix a family v and a parent configuration π with $P_p(\mathrm{pa}_v{=}\pi) > 0$. The row $P_p(\mathrm{ch}_v \mid \mathrm{pa}_v{=}\pi)$ is a conditional table attainable by the joint p, and p satisfies every sentence of $\mathcal{L}$ — in particular every

sentence whose atom scope lies inside $\mathrm{ch}_v \cup \mathrm{pa}_v$. But $K_{v,\pi}$ was defined as the set of rows attainable by *some* joint satisfying exactly those in-scope sentences. Hence $P_p(\mathrm{ch}_v \mid \mathrm{pa}_v{=}\pi) \in K_{v,\pi}$: the local set is a *relaxation* of the projection, so it contains it. For a configuration with $P_p(\mathrm{pa}_v{=}\pi) = 0$ the row is unconstrained by p and may be chosen to be any element of $K_{v,\pi}$ (non-empty, since p itself witnesses feasibility of the in-scope sentences) without changing the product.

Thus p is exhibited as a product $\prod_v \prod_\pi$ of rows drawn from the respective $K_{v,\pi}$, which is precisely membership in $\mathcal{D}_{\mathrm{SE}}$ as defined in (5). The bound inclusion follows because minimizing and maximizing a fixed functional over a larger set yields a wider interval. □

Proposition 2 (Sufficiency for equality)**.** *Let $\mathcal{L}$ be a chain-graph LCN with local sets built by `method="linear"`. Suppose*

(A) no Gap A: *for every family v and parent configuration π, $K_{v,\pi} = K^{\mathrm{true}}_{v,\pi}$, the projection of $\mathcal{D}_{\mathrm{LCN}}$ onto that row; and*

(B) full realizability: *every sentence of $\mathcal{L}$ is family-realizable in some family (Definition 2).*

Then $\mathcal{D}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$. Single-connectedness of the primal graph is not required.

Proof. Proposition 1 gives $\mathcal{D}_{\mathrm{LCN}} \subseteq \mathcal{D}_{\mathrm{SE}}$, so only $\mathcal{D}_{\mathrm{SE}} \subseteq \mathcal{D}_{\mathrm{LCN}}$ needs proof. Let $q = \prod_v \prod_\pi P_{v,\pi}$ with each $P_{v,\pi} \in K_{v,\pi}$ be an arbitrary element of $\mathcal{D}_{\mathrm{SE}}$. We verify that q satisfies both families of constraints in (3).

LMC equalities. By construction q is a product of the form (4) over the families of the chain graph. By the converse direction of the chain-graph factorization theorem (Lauritzen, Thm. 3.34: factorization implies the global Markov property of the chain graph, and this direction needs no positivity assumption), q satisfies every conditional independence entailed by the chain graph — in particular every LMC assertion, since the LMC is by definition read off that same graph. Hence $h_\iota(q) = 0$ for all ι. Note this step is where the "loopiness contributes nothing" claim comes from: a product over the chain-graph families satisfies the LMC whether or not the graph is singly connected.

Sentences. Let s be a sentence. By (B) it is family-realizable in some family v, so by Definition 2 the value $g_s(q)$ is determined by the conditional table of q at v alone. That table is the collection of rows $\{P_{v,\pi}\}_\pi$, and by (A) each $P_{v,\pi} \in K^{\mathrm{true}}_{v,\pi}$, i.e. each row is realized by *some* $p^{(\pi)} \in \mathcal{D}_{\mathrm{LCN}}$. It remains to see that this forces $g_s(q) \in [\ell_s, u_s]$. Two cases, matching the two clauses of Definition 2.

If s has form (R2), then $g_s(q) = P_{v,\pi^\star}(\varphi_{\pi^\star})$ is the *single* row $\pi^\star$. That row equals the corresponding row of some $p^{(\pi^\star)} \in \mathcal{D}_{\mathrm{LCN}}$; by Definition 2 again (applied to $p^{(\pi^\star)}$, whose table has this same row) $g_s(p^{(\pi^\star)}) = g_s(q)$, and $p^{(\pi^\star)} \in \mathcal{D}_{\mathrm{LCN}}$ satisfies s. Hence $g_s(q) \in [\ell_s, u_s]$.

If s has form (R1), then $\mathrm{pa}_v = \varnothing$, so the family has the single (empty) parent configuration and its table is the whole marginal $P(\mathrm{ch}_v) = P_{v,\varnothing}$. The same argument applies verbatim with $\pi^\star$ the empty configuration.

In both cases the key point is that realizability makes g_s a function of *one row*, and a row certified by (A) to be realized inside $\mathcal{D}_{\mathrm{LCN}}$ carries its constraint value with it — so the free, independent choice of the *other* rows and families cannot disturb it. Therefore every sentence holds at q, giving $q \in \mathcal{D}_{\mathrm{LCN}}$. □

Remark 5 (Why there is no biconditional over "every atom")**.** The companion note `strong_extension_exactness.tex` states its main theorem as an *iff*: $\mathcal{D}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$ — equivalently, Credal VE is exact for every atom — iff the primal graph is singly connected and every sentence is family-realizable. Two problems make that formulation untenable, and we therefore state Propositions 1–2 plus witness examples instead.

First, set-strictness does not imply marginal looseness. On `examples/d4_biting.lcn` the cross-family sentence $P(B \wedge C) \leq 0.05$ is non-realizable and the free product genuinely violates it (reaching $P(B \wedge C) \approx 0.92$), so $\mathcal{D}_{\mathrm{SE}} \supsetneq \mathcal{D}_{\mathrm{LCN}}$. Yet all three unconditional marginals are *exact*: measured, Credal VE returns $P(A) = [0.40, 0.60]$, $P(B) = [0.04, 0.72]$, $P(C) = [0.08, 0.80]$, matching `ExactInference(global)` and `CredalJT` endpoint for endpoint. The extra joints in $\mathcal{D}_{\mathrm{SE}} \setminus \mathcal{D}_{\mathrm{LCN}}$ simply do not attain any singleton-marginal extremum. So "$\mathcal{D}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$" is strictly stronger than "every atom is exact", and the two cannot be equivalent.

Second, single-connectedness is not necessary — see Example 5.

What *is* true, and is proved above, is sufficiency; the necessity of hypothesis (B) is established by the witnesses of Examples 3 and 4, which show that dropping realizability makes specific marginals strictly loose. That is a complete and honest account of the boundary.

Example 5 (A loopy LCN on which Credal VE is exact)**.** Single-connectedness is not necessary for exactness, and the topology table of earlier versions of this note — which marked Credal VE "outer only" on `ktree-fr` *because* that class is loopy — was wrong on this point. Consider `examples/ktree_fr_k2_n4_example.lcn` (n=4, k=2):

$$s_0\colon\ 0.084382 \leq P(x_1) \leq 0.684382 \qquad s_2\colon\ 0.092785 \leq P(x_3 \mid x_1 \wedge x_2) \leq 0.692785$$
$$s_1\colon\ 0.01 \leq P(\neg x_2 \mid \neg x_1) \leq 0.572656 \qquad s_3\colon\ 0.068242 \leq P(x_0 \mid x_1 \wedge x_3) \leq 0.668242$$

The seed 3-clique is $\{x_1, x_2, x_3\}$ and x_0 attaches to the 2-clique $\{x_1, x_3\}$, so the moralized graph contains the triangle $x_1x_2x_3$: the instance is **loopy**, decidedly not singly connected. It is also fully realizable (each ψ is a full conjunction over its family's parents; s_0 is a root marginal). Measured, with all eight SCIP solves `confirmed` at gap 0:

engine	$P(x_0)$	$P(x_1)$	$P(x_2)$	$P(x_3)$
`ExactInference` (global)	$[0, 1]$	$[0.0844, 0.6844]$	$[0.1349, 0.9968]$	$[0, 1]$
`CredalJT` (D5)	$[0, 1]$	$[0.0844, 0.6844]$	$[0.1349, 0.9968]$	$[0, 1]$
Credal VE	$[0, 1]$	$[0.0844, 0.6844]$	$[0.1349, 0.9968]$	$[0, 1]$

Credal VE is *exact on every atom* of a loopy LCN. Moreover `method="linear-tight"` (D1, a pure Gap-A closer) returns identical numbers, so there is no residual Gap A either. This is consistent with Proposition 2, whose hypotheses (A) and (B) hold here and which never mentions the topology; and it refutes the inference "loopy ⇒ Gap B". The correct statement is that loopiness alone does not create Gap B: a non-realizable sentence, or an open Gap A, does.

Remark 6 (What loopiness *does* cost)**.** Loopiness is not free — but its price is paid in *cost*, not accuracy. On the five-atom `ktree_fr_k3_n5_1.lcn` of Example 2, the credal network builds essentially instantly and is tiny ($8 \cdot 8 \cdot 4 \cdot 1 \cdot 2 = 512$ vertex combinations across the five families), yet Credal VE did not terminate within 200 s: it stalls inside `Potential.prune` during elimination. `CredalJT` solves the same instance in about a second. So on bounded-treewidth loopy classes the practical recommendation — prefer `CredalJT` — stands, but the reason is tractability, not looseness.

4 The credal-network engines

All engines in this section consume the same compiled `CredalNetworkVertices`; they differ only in how they propagate the enumerated extreme points. Throughout, `method="linear"` is assumed (Remark 3).

4.1 Two optimization lemmas

Both results below rest on the same geometric fact, which we state carefully because the form usually quoted ("a multilinear objective attains its extremum at a product of vertices") is not the form that is needed: the read-off is a *ratio*, not a multilinear function.

Lemma 3 (Block-affine vertex attainment)**.** *Let $K = \prod_{j=1}^{m} K_j$ with each $K_j \subseteq \mathbb{R}^{d_j}$ compact and convex, and let $F : K \to \mathbb{R}$ be continuous and* affine in each block separately *(fixing all blocks but j, the restriction is affine on K_j). Then F attains its maximum and its minimum on $\prod_j \mathrm{ext}(K_j)$.*

Proof. K is compact and F continuous, so a maximizer $t^\star = (t_1^\star, \dots, t_m^\star)$ exists. We show it can be moved to a product of extreme points without decreasing F. Fix j and freeze the other blocks at $t^\star$. The map $t_j \mapsto F(\dots, t_j, \dots)$ is affine on the compact convex K_j, hence in particular convex; by the Bauer maximum principle a convex continuous function on a compact convex set attains its maximum at an extreme point, so there is $\hat{t}_j \in \mathrm{ext}(K_j)$ with $F(\dots, \hat{t}_j, \dots) \geq F(t^\star)$. Replace $t_j^\star$ by $\hat{t}_j$; the value has not decreased. Iterating over $j = 1, \dots, m$ (each step freezes the blocks already replaced at extreme points, and affinity in block j is unaffected by the values of the others) yields a point of $\prod_j \mathrm{ext}(K_j)$ with value $\geq F(t^\star)$, hence equal to the maximum. For the minimum apply the same argument to $-F$, which is also block-affine. □

Lemma 4 (Block-fractional vertex attainment)**.** *Let $K = \prod_{j=1}^{m} K_j$ with each K_j compact convex, and let $N, D : K \to \mathbb{R}$ be continuous and affine in each block separately, with $D(t) > 0$ for all $t \in K$. Then $F = N/D$ attains its maximum and its minimum on $\prod_j \mathrm{ext}(K_j)$.*

Proof. F is continuous on the compact K (as $D > 0$), so extrema exist. Fix j, freeze the other blocks, and consider $\phi(t_j) = N(t_j)/D(t_j)$ on K_j, with N, D affine in t_j and $D > 0$. Take any segment $t_j(\theta) = (1-\theta)a + \theta b$, $\theta \in [0,1]$, in K_j. Then $\theta \mapsto N(t_j(\theta))$ and $\theta \mapsto D(t_j(\theta))$ are affine in θ, so $\phi(t_j(\theta)) = \frac{n_0 + n_1\theta}{d_0 + d_1\theta}$ with $d_0 + d_1\theta > 0$ on $[0,1]$. Its derivative is $\frac{n_1 d_0 - n_0 d_1}{(d_0 + d_1\theta)^2}$, whose sign is constant on $[0,1]$: ϕ is therefore *monotone* along every segment. A function monotone along all segments of a convex set attains both its maximum and its minimum at extreme points — for the maximum: given any $t_j \in K_j$, write it (Krein–Milman, finite dimension: Minkowski's theorem) as a convex combination of extreme points and note that repeatedly moving along a segment towards whichever endpoint does not decrease ϕ terminates at an extreme point with value $\geq \phi(t_j)$; symmetrically for the minimum. So there is $\hat{t}_j \in \mathrm{ext}(K_j)$ not decreasing F. Iterate over the blocks as in Lemma 3. □

Remark 7 (Why the fractional form is the one needed)**.** Credal VE's read-off (`cve.py`, Step 6) normalizes each surviving function, $f \mapsto f / \sum_a f[a]$, and reports the componentwise extrema of the normalized vectors. So even for an *unconditional* marginal the reported quantity is a ratio of two block-affine functions, and Lemma 4 — not Lemma 3 — is what applies. For a *conditional* query $P(a \mid e) = P(a, e)/P(e)$ the same lemma applies with N, D the two block-affine marginals. In both cases the hypothesis $D > 0$ is essential and is *not* automatic: the implementation skips offending functions outright (`if total <= 0: continue`). We therefore carry $\min_{P \in \mathcal{D}_{\mathrm{SE}}} P(e) > 0$ as an explicit hypothesis.

4.2 Credal VE

Credal VE (`cn/cve.py`) computes each atom's bounds by a single-query bucket elimination, each time recomputing an elimination order in which the target is eliminated *last*. Potentials are *sets* of functions; `combine` takes elementwise products pairwise, `marginalize` sums out a variable, and `Potential.prune` discards *dominated* functions (f is dominated if some g in the same potential satisfies $g \geq f$ componentwise, with strict inequality somewhere).

Lemma 5 (Prune monotonicity)**.** *Let Φ be any composition of* `combine` *operations (with fixed non-negative co-factors) and* `marginalize` *operations, mapping functions on a scope S to functions on a scope S'. If $g \geq f$ pointwise on S then $\Phi(g) \geq \Phi(f)$ pointwise on S'. Consequently, for every state s' of S',*

$$\max_{f \in \mathcal{P}} \Phi(f)[s'] = \max_{f \in \mathrm{prune}(\mathcal{P})} \Phi(f)[s'].$$

Proof. It suffices to check the two primitive operations, since Φ is their composition. For `combine` with a fixed $h \geq 0$: $g \geq f$ implies $g \cdot h \geq f \cdot h$ pointwise, as multiplying an inequality between non-negative numbers by a non-negative number preserves it. For `marginalize` over a variable x: $g \geq f$ pointwise implies $\sum_x g \geq \sum_x f$ pointwise in the remaining variables, as a sum of termwise inequalities. Composing preserves the property.

For the second claim: `prune` removes f only when some g in the same potential dominates it, and by the above $\Phi(g)[s'] \geq \Phi(f)[s']$, so the removed f cannot be the unique argmax at any state s'. Since `prune` never removes all functions, and domination is transitive, the maximum over the pruned set is unchanged. □

Theorem 1 (Credal VE is exact over the strong extension)**.** *Let $\mathcal{L}$ be a chain-graph LCN with* `method="linear"` *local sets, and run Credal VE with* `coupling="off"` *and exact pruning (ϵ=0) on an* unconditional *marginal query $P(a)$. Then Credal VE returns exactly the strong-extension interval,*

$$[\underline{q}_{\mathrm{VE}}(a), \overline{q}_{\mathrm{VE}}(a)] = \Big[\min_{P \in \mathcal{D}_{\mathrm{SE}}} P(a),\ \max_{P \in \mathcal{D}_{\mathrm{SE}}} P(a)\Big],$$

and hence, by Proposition 1, a valid outer bound w.r.t. $\mathcal{D}_{\mathrm{LCN}}$; under the hypotheses of Proposition 2 it is exact w.r.t. $\mathcal{D}_{\mathrm{LCN}}$.

Proof. Write $\mathcal{V} = \prod_v \prod_\pi \mathrm{ext}(K_{v,\pi})$ for the set of vertex selections, so that each $\tau \in \mathcal{V}$ determines a product joint $P_\tau \in \mathcal{D}_{\mathrm{SE}}$.

Step 1: every function formed is a vertex product, and all have total mass one. By induction over the elimination schedule. Initially each family's potential is the set of its enumerated local extreme points, indexed by the rows of that family. `combine` of two potentials forms all pairwise elementwise products, and `marginalize` sums a variable out; hence every function appearing at any stage is, for some $\tau \in \mathcal{V}$, the marginal of P_τ on the current scope — this is exactly the standard bucket-elimination identity for the precise network P_τ, applied uniformly across τ. In particular, since the query is eliminated *last*, when the schedule finishes the surviving potential has scope $\{a\}$ and its functions are precisely $\{(P_\tau \restriction \{a\})\}_{\tau \in \mathcal{T}}$ for the set $\mathcal{T} \subseteq \mathcal{V}$ of selections that survived pruning. Each such function is a probability distribution on $\{a\}$: it has total mass $\sum_a P_\tau(a) = 1$, because P_τ is a joint probability distribution and no evidence indicator was multiplied in (this is where "unconditional" is used).

Step 2: normalization is the identity, so pruning is harmless. With no evidence, every final function has total mass 1 by Step 1, so the read-off $f \mapsto f / \sum_a f[a]$ leaves each function unchanged. Hence the reported bounds are the componentwise extrema $\min_{\tau \in \mathcal{T}} P_\tau(a)$ and $\max_{\tau \in \mathcal{T}} P_\tau(a)$ of the surviving functions. By Lemma 5 applied to the composition Φ of the operations performed after each pruning step, these componentwise extrema over $\mathcal{T}$ coincide with those over the full $\mathcal{V}$: $\min_{\tau \in \mathcal{T}} P_\tau(a) = \min_{\tau \in \mathcal{V}} P_\tau(a)$ and likewise for max.

Step 3: the vertex extremum is the $\mathcal{D}_{\mathrm{SE}}$ extremum. The map $\tau \mapsto P_\tau(a)$ extends to $\prod_v \prod_\pi K_{v,\pi} \to \mathbb{R}$, $(P_{v,\pi}) \mapsto \big(\prod_v \prod_\pi P_{v,\pi}\big)(a)$, which is affine in each block $P_{v,\pi}$ separately: expanding the product, each block's table appears to degree exactly one. By Lemma 3 the extrema

of this map over the product of the (compact, convex) $K_{v,\pi}$ are attained at $\mathcal{V}$. Since $\mathcal{D}_{\mathrm{SE}}$ is precisely the image of that product under the map,

$$\min_{P\in\mathcal{D}_{\mathrm{SE}}} P(a) = \min_{\tau\in\mathcal{V}} P_\tau(a), \qquad \max_{P\in\mathcal{D}_{\mathrm{SE}}} P(a) = \max_{\tau\in\mathcal{V}} P_\tau(a).$$

Chaining Steps 2 and 3 gives the claim. The final sentence follows from Propositions 1 and 2. □

Remark 8 (Conditional queries: a genuine gap)**.** Theorem 1 is deliberately restricted to unconditional marginals, and the restriction cannot simply be lifted. With evidence, an indicator potential is multiplied in, so a final function has total mass $P_\tau(e)$, which *varies with* τ. Normalization is then a τ*-dependent* rescaling and Step 2 breaks: the read-off ratio is not monotone under componentwise domination. Concretely, over states (a=1, a=0) the function $g = (1, 1)$ dominates $f = (1, 0)$, so `prune` discards f — yet f normalizes to read-off 1.0 and g to 0.5. Pruning can therefore discard the function carrying the extremal conditional, yielding an *inner* (unsound) interval. This is the same mechanism that makes Credal CTE inner (Section 4.3); the difference is that CTE additionally shares one order across all queries, so it is exposed even without evidence.

Measured: the hazard did not materialize on the instances tested here. Monkey-patching `Potential.prune` to the identity leaves Credal VE's output bit-identical on `d4_biting`, on the collider witness of Example 3, and on the two-atom witness of Example 4. So the theoretical gap is real but not currently observed; we state it rather than assume it away.

Example 6 (Credal VE against three references)**.** On `examples/d4_biting.lcn` (three atoms $A \to B$, $A \to C$, plus the cross-family sentence $P(B \wedge C) \leq 0.05$), Credal VE returns $P(A) = [0.40, 0.60]$, $P(B) = [0.04, 0.72]$, $P(C) = [0.08, 0.80]$, agreeing with `ExactInference(global)` and `CredalJT` to four decimals. Here $\mathcal{D}_{\mathrm{SE}} \supsetneq \mathcal{D}_{\mathrm{LCN}}$ (the free product violates the cap) yet the marginals coincide — the phenomenon of Remark 5. On the loopy Example 5 Credal VE is likewise exact on all four atoms; on the non-realizable Examples 3 and 4 it returns the strictly wider strong-extension interval, as Theorem 1 predicts.

4.3 Credal CTE: not exact, and not even outer

Credal CTE (`cn/ccte.py`) is the two-pass (collect + distribute) cluster-tree engine: one *shared* elimination order produces all marginals at once, deliberately *not* augmented per query. That single design decision costs it both exactness and soundness.

Proposition 3 (CTE can be strictly inner)**.** *With* `coupling="off"` *and exact pruning, Credal CTE's interval can be strictly contained in the* $\mathcal{D}_{\mathrm{SE}}$ *interval; in particular it is* not *in general a valid outer bound w.r.t.* $\mathcal{D}_{\mathrm{LCN}}$.

Proof. It suffices to exhibit one instance, since the claim is a non-containment. On the shipped `d4_biting` instance the measured outputs are

engine	$P(A)$	$P(B)$	$P(C)$
`ExactInference` (global, certified)	$[0.40, 0.60]$	$[\mathbf{0.04}, \mathbf{0.72}]$	$[0.08, 0.80]$
Credal VE = `CredalJT` = Interval BP = ApproxLP	$[0.40, 0.60]$	$[0.04, 0.72]$	$[0.08, 0.80]$
Credal CTE	$[0.40, 0.60]$	$[\mathbf{0.05}, \mathbf{0.65}]$	$[0.08, 0.80]$

Since $[0.05, 0.65] \subsetneq [0.04, 0.72]$ and the latter is the exact $\mathcal{D}_{\mathrm{LCN}}$ (and here also $\mathcal{D}_{\mathrm{SE}}$) interval for $P(B)$, CTE's interval is strictly inner and excludes feasible marginals.

The mechanism is the interaction of Lemma 5's hypothesis with the shared order. Credal VE eliminates the query last, so by Step 2 of Theorem 1 the normalization at read-off is trivial and

dominance pruning is harmless. CTE eliminates B mid-schedule, at which point the function carrying the extremal $P(B)$ is dominated *at that intermediate scope* and is pruned — but domination at a marginalized scope does not imply domination of the eventual normalized read-off, exactly as in Remark 8. The extremal vertex combination is therefore discarded before it can be read off. □

Remark 9 (A correction to the companion notes)**.** `tighter_approximation.tex` lists Credal CTE as "exact (ϵ=0); outer if $\epsilon > 0$", while `ccte_exactness.tex` reports it as strictly inner. The measurement in Proposition 3 settles the disagreement in favour of `ccte_exactness.tex`. CTE *is* exact on a clean singly-connected, cross-family-sentence-free LCN (there the shared schedule is a genuine clique tree and the surviving dominant function also dominates for every downstream read-off), but that hypothesis must be stated: it is not exact at ϵ=0 in general.

4.4 Interval BP

Interval BP (`cn/ibp.py`) is interval message passing over the credal network's factor graph. With `method="interval"` it propagates per-state intervals; with `method="variational"` it runs mean-field over a finite sample of vertex combinations and is *inner* by construction (it evaluates a subset of $\mathcal{D}_{\mathrm{SE}}$).

Theorem 2 (Interval BP is outer, and exact on trees)**.** *With `method="interval"`:*

(i) *For every $P \in \mathcal{D}_{\mathrm{SE}}$, every edge and every iteration, P's marginal on the message's variable lies in the message interval. Hence the returned interval is a valid* outer *bound w.r.t. $\mathcal{D}_{\mathrm{SE}}$, on any topology.*

(ii) *If the factor graph is a tree, the converged intervals equal the $\mathcal{D}_{\mathrm{SE}}$ interval for every atom; combined with Proposition 2 they are then exact w.r.t. $\mathcal{D}_{\mathrm{LCN}}$.*

Proof. (i) Induction on the iteration count t, simultaneously over all edges. At $t = 0$ every message is $[0, 1]$, which contains every marginal. Assume the claim at t and fix $P = \prod_v \prod_\pi P_{v,\pi} \in \mathcal{D}_{\mathrm{SE}}$. A *variable-to-factor* message is the intersection of the other incoming factor-to-variable intervals; by the inductive hypothesis each of those contains P's marginal of that variable, so their intersection does too (an intersection of sets each containing a common point contains it). A *factor-to-child* update ranges over all local extreme points of that factor's credal set and over all corner distributions of the incoming messages, and returns the elementwise min/max of the resulting marginal. Since P's own local table at this factor is a convex combination of that factor's extreme points, and P's incoming marginals lie in the incoming intervals by induction, P's marginal of the child is a convex combination of values that the update ranged over, hence lies between the min and max it reports. A conditional factor sends the vacuous $[0, 1]$ upward, which excludes nothing. So the claim holds at $t + 1$. Taking the final intersection over incident factors preserves containment, giving the outer bound.

(ii) Suppose the factor graph is a tree. We show the converged interval is also *attained*, which with (i) gives equality. Root the tree at the query atom a and induct on subtree depth. For a leaf factor the update optimizes over that factor's extreme points with no incoming constraint, so both endpoints are attained by an actual vertex selection. For an internal factor f with children subtrees $T_1, \ldots, T_k$: because the graph is a tree, the subtrees are vertex-disjoint apart from f and share no variable, so the vertex selections in distinct T_i are *independent* degrees of freedom of $\mathcal{D}_{\mathrm{SE}}$ — no selection is constrained twice. By induction each incoming message endpoint is attained by some selection within its subtree; combining those selections (legitimate, by independence) with the extreme point of f's own set that the update chose yields a single $\tau \in \mathcal{V}$ attaining the reported

endpoint. Since the message endpoints are attained rather than merely containing, no slack is introduced, and at the root the reported interval equals $\{P_\tau(a)\}$'s extremes, which by Step 3 of Theorem 1 is the $\mathcal{D}_{\mathrm{SE}}$ interval. On a loopy graph the independence step fails — a selection reachable through two paths would be constrained twice while the interval algebra treats the two messages as independent — which is precisely the source of loop slack. □

Remark 10 (A residual architectural limitation, measured)**.** Theorem 2(ii) concerns unconditional marginals. Interval BP's intersection message algebra performs no Bayesian *upward* update, so evidence on a child does not shift a parent's posterior. On the two-atom chain of Example 4 (without `M1`), querying $P(x_0 \mid x_1{=}1)$:

engine	$P(x_0 \mid x_1{=}1)$
Interval BP (`interval`)	$[0.30, 0.50]$ — the *unchanged prior*
Credal VE	$[0.1765, 0.3333]$
`CredalJT` (D5)	$[0.0968, 0.4000]$

Interval BP simply returns $P(x_0)$: the evidence has no effect. For conditional queries with evidence on descendants, use Credal VE or `CredalJT`. (That the latter two also disagree here is expected — Credal VE bounds $\mathcal{D}_{\mathrm{SE}}$ conditionally, subject to Remark 8, while `CredalJT` targets $\mathcal{D}_{\mathrm{LCN}}$.)

Remark 11 (Shipped status: two bugs, both fixed)**.** Interval BP previously carried two message-passing bugs that made the interval mode *inner* and unsound, contradicting Theorem 2(i): a spurious normalized-likelihood upward message (which collapsed an unconstrained root to $[0.5, 0.5]$) and an invalid component-wise corner enumeration. Both are corrected. Verified: on the two-atom chain the engine now returns $P(x_0) = [0.30, 0.50]$ (was $[0.5, 0.5]$) and on `d4_biting` $P(B) = [0.04, 0.72]$ (was $[0.05, 0.65]$).

4.5 ApproxLP

ApproxLP (`cn/approxlp.py`) solves the multilinear program by coordinate descent: initialize every local table to the *center* of its credal set (the mean of the enumerated extreme points); then sweep, and for each (family, parent-configuration) block hold the others fixed, enumerate that block's extreme points, evaluate the objective exactly by variable elimination on the resulting *precise* Bayesian network, and greedily commit the best; repeat to convergence.

Theorem 3 (ApproxLP is inner w.r.t. the strong extension)**.** *Run ApproxLP with* `coupling="off"` *on a query whose denominator is positive throughout (for an unconditional marginal this is automatic). Then*

$$\min_{P \in \mathcal{D}_{\mathrm{SE}}} q(P) \;\le\; \underline{q}_{\mathrm{ALP}} \;\le\; \overline{q}_{\mathrm{ALP}} \;\le\; \max_{P \in \mathcal{D}_{\mathrm{SE}}} q(P),$$

i.e. ApproxLP's interval is contained in the $\mathcal{D}_{\mathrm{SE}}$ interval. It is therefore not in general a valid outer bound w.r.t. $\mathcal{D}_{\mathrm{SE}}$.

Proof. Step 1: every state visited is a point of $\mathcal{D}_{\mathrm{SE}}$. By induction over the sweep. The initial state assigns each block the center of $K_{v,\pi}$, i.e. the arithmetic mean of its extreme points; since $K_{v,\pi}$ is convex and its extreme points belong to it, the mean lies in $K_{v,\pi}$. For the inductive step, a commit replaces one block's value by an extreme point of that same $K_{v,\pi}$, which lies in $K_{v,\pi}$ by definition; all other blocks are untouched. Hence after every commit the state is a tuple $(P_{v,\pi})$ with $P_{v,\pi} \in K_{v,\pi}$ for all (v, π) — note that this is exactly membership of the assembled product in $\mathcal{D}_{\mathrm{SE}}$ as defined by (5), because (5) allows each block to be chosen independently (Remark 2).

Mixed states, with some blocks at centers and others at vertices, are therefore admissible members of $\mathcal{D}_{\mathrm{SE}}$; this is the point at which the row-wise reading of (5) is load-bearing.

Step 2: the objective is evaluated exactly at each visited state. At a visited state the network is a *precise* Bayesian network P_τ (every block has a single table), and the evaluation runs standard variable elimination on the factorization (4). Variable elimination computes marginals of a precise factorized distribution exactly (it is the distributive law applied to the sum over eliminated variables), and for a conditional query it forms the ratio of two such marginals, which is exact provided the denominator $P_\tau(e) > 0$ — the hypothesis. Hence every value ApproxLP records equals $q(P)$ for that specific $P \in \mathcal{D}_{\mathrm{SE}}$.

Step 3: conclusion. By Steps 1–2 the set of values ApproxLP ever records is a subset of $\{q(P) : P \in \mathcal{D}_{\mathrm{SE}}\}$. Its reported lower bound is the value at the terminal state of the `sense="min"` descent and its reported upper bound that of the `sense="max"` descent; both are elements of that value set. A minimum over a subset is $\geq$ the minimum over the whole set, and a maximum over a subset is $\leq$ the maximum, giving $\min_{\mathcal{D}_{\mathrm{SE}}} q \leq \underline{q}_{\mathrm{ALP}}$ and $\overline{q}_{\mathrm{ALP}} \leq \max_{\mathcal{D}_{\mathrm{SE}}} q$. Finally $\underline{q}_{\mathrm{ALP}} \leq \overline{q}_{\mathrm{ALP}}$: both descents start from the *same* center state τ_0, and each is monotone in its own direction (a commit is made only if it improves the objective), so $\underline{q}_{\mathrm{ALP}} \leq q(\tau_0) \leq \overline{q}_{\mathrm{ALP}}$. □

Proposition 4 (Tightness is an assumption about the landscape, not a topological condition)**.** *Suppose (a) $\mathcal{D}_{\mathrm{SE}} = \mathcal{D}_{\mathrm{LCN}}$ (e.g. under the hypotheses of Proposition 2), and (b) for each sense the coordinate descent terminates at a* global *optimum of q over $\mathcal{D}_{\mathrm{SE}}$. Then ApproxLP's interval equals the exact $\mathcal{D}_{\mathrm{LCN}}$ interval.*

Proof. Immediate from Theorem 3: by (b) the two terminal values are the global $\mathcal{D}_{\mathrm{SE}}$ extrema, so the chain of inequalities in Theorem 3 collapses to equalities, and by (a) the $\mathcal{D}_{\mathrm{SE}}$ interval is the $\mathcal{D}_{\mathrm{LCN}}$ interval. □

Remark 12 (Honesty about hypothesis (b))**.** Hypothesis (b) is an assumption about the optimization landscape, not a checkable structural condition, and we state it as such. The companion note's version of this result assumes instead that "the per-family linearizations expose their global optimum to a greedy sweep from the center", and then proves tightness by invoking that assumption — which is the conclusion. No structural sufficient condition for (b) is known; the multilinear program is nonconvex, so a sweep can stall where no single-block move improves but a coordinated multi-block jump would. The practical substitute is the bracket of Corollary 1.

Corollary 1 (Two-sided bracket and stall detection)**.** *Run ApproxLP and Credal VE on the* same *enumerated vertices for an unconditional marginal. Then $[\underline{q}_{\mathrm{ALP}}, \overline{q}_{\mathrm{ALP}}] \subseteq [\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}] = [\underline{q}_{\mathrm{VE}}, \overline{q}_{\mathrm{VE}}]$. If the two intervals coincide, the common value is certified to be the exact $\mathcal{D}_{\mathrm{SE}}$ bound; if they differ, the difference is exactly the coordinate-descent stall.*

Proof. The inclusion is Theorem 3 and the equality is Theorem 1. If the endpoints coincide then ApproxLP attained the $\mathcal{D}_{\mathrm{SE}}$ extremum, certifying it; otherwise Theorem 3's inequality is strict, which by Step 3 of its proof happens only when the terminal state is not a global optimum. □

Example 7 (ApproxLP is inner w.r.t. $\mathcal{D}_{\mathrm{SE}}$ yet *outer* w.r.t. $\mathcal{D}_{\mathrm{LCN}}$)**.** This is why Definition 1 insists on naming the reference set. On `d4_biting` ApproxLP returns $P(B) = [0.04, 0.72]$ — coinciding with Credal VE, so by Corollary 1 the $\mathcal{D}_{\mathrm{SE}}$ bound is certified and the inner gap is zero. But on the two-atom witness of Example 4 *with* `M1`, ApproxLP returns $P(x_1) = [0.40, 0.68]$ while the exact $\mathcal{D}_{\mathrm{LCN}}$ bound is $[0.45, 0.55]$: here ApproxLP strictly *contains* the truth, i.e. it is an *outer* bound w.r.t. $\mathcal{D}_{\mathrm{LCN}}$. Both facts are consistent — ApproxLP is inner w.r.t. $\mathcal{D}_{\mathrm{SE}}$, and $\mathcal{D}_{\mathrm{LCN}} \subsetneq \mathcal{D}_{\mathrm{SE}}$ here, with the Gap B looseness dominating the (zero) descent slack. The unqualified assertion "ApproxLP

is not a valid outer bound" is therefore false as stated; the correct statement is that it carries no outer guarantee w.r.t. $\mathcal{D}_{\mathrm{SE}}$, and its position relative to $\mathcal{D}_{\mathrm{LCN}}$ depends on which effect dominates.

4.6 Credal IJGP

Credal IJGP (`cn/ijgp.py`) runs iterative join-graph propagation with a mini-bucket `i_bound` capping cluster size.

Proposition 5 (IJGP at large i-bound)**.** *If `i_bound` $\geq$ the treewidth of the credal network's moral graph, no mini-bucket partitioning is forced, the join graph is a junction tree, and Credal IJGP coincides with exact elimination over the enumerated vertices — hence is exact over $\mathcal{D}_{\mathrm{SE}}$. For smaller `i_bound` it is an outer-style approximation whose slack the i-bound trades against a cost of 2^i per cluster.*

Proof. At `i_bound` $\geq$ treewidth every bucket fits within the cap, so the mini-bucket splitting step is vacuous and the join graph produced is the junction tree of the elimination order, which has the running-intersection property and no cycle. Message passing on a cycle-free join graph is exact elimination (the same argument as Theorem 2(ii), one level up at cluster rather than variable granularity: disjoint subtrees supply independent vertex selections, so each message endpoint is attained). By Step 3 of Theorem 1 the vertex extremum is the $\mathcal{D}_{\mathrm{SE}}$ extremum. When `i_bound` is below the treewidth, a bucket is split into mini-buckets whose messages are propagated independently, which duplicates the influence of shared selections and introduces the usual loopy slack. □

Example 8 (IJGP on `d4_biting`)**.** Measured: $P(A) = [0.40, 0.60]$, $P(B) = [0.04, 0.72]$, $P(C) = [0.08, 0.80]$ at both `i_bound`= 1 and `i_bound`= 3 — exact, matching the certified oracle. The instance's moral graph is the single clique $\{A, B, C\}$ of treewidth 2, and even the i-bound-1 run happens not to lose anything here; on larger loopy instances the i-bound is the tightness dial of Proposition 5.

5 CredalJT: exact at treewidth cost

`CredalJT` (`cn/junction_nlp.py`, scheme D5) does not compile to a credal network at all: it builds one junction tree from the chain-graph moral graph, gives each cluster a variable vector over the joint of its atoms, ties adjacent clusters by separator-consistency equalities, hosts every sentence and LMC equality on a cluster containing its atoms, and optimizes the read-off with SCIP. Its cost is $\sum_C 2^{|C|} = O(n\, 2^{w+1})$ — exponential in the *treewidth* w, not in n.

Definition 3 (Cluster-feasible set, hosting, separator-entailment, read-off)**.** Let T be a clique tree over clusters $\mathcal{C}$, built by the standard moralize $\to$ triangulate $\to$ clique-tree pipeline, and let G^* be the chordal graph whose maximal cliques are the clusters (two atoms adjacent iff they co-occur in a cluster).

- The *cluster-feasible set* is

$$\mathcal{G} = \big\{ (q_C)_{C\in\mathcal{C}} \;:\; q_C \in \Delta(2^{|C|}) \text{ for each } C; \\ q_C \restriction S = q_D \restriction S \text{ for every edge } (C, D) \in T,\ S = C \cap D; \\ \text{every hosted constraint holds on its host cluster} \big\}.$$

- A constraint c is *hosted* if some cluster $C_h \supseteq \mathrm{scope}(c)$ exists; its row is then imposed on q_{C_h}'s marginal.

- An LMC assertion $(X \perp\!\!\!\perp Y \mid S)$ is *separator-entailed* if S separates X from Y in G^*.
- For a query atom a and evidence e lying together in a cluster C_a, the *read-off* is $\rho_a(q) = \sum_{i:a=1} q_{C_a}[i]$ (no evidence), or $\rho_a(q) = \big(\sum_{i:a=1,e} q_{C_a}[i]\big)/\big(\sum_{i:e} q_{C_a}[i]\big)$ (with evidence).
- $\pi(p) = (p \upharpoonright C)_{C \in \mathcal{C}}$ is the *projection* of a joint onto the clusters.

The reconstruction direction needs the following lemma, whose usual statement contains two defects we must repair: it is normally asserted with a *uniqueness* claim that is false in the presence of zero separator cells, and the division in the product formula is left unaddressed.

Lemma 6 (RIP reconstruction, with zeros)**.** *Let T be a clique tree with the running-intersection property over clusters $\mathcal{C}$, and let $(q_C) \in \prod_C \Delta(2^{|C|})$ be separator-consistent: $q_C \upharpoonright S = q_D \upharpoonright S$ for every edge (C, D) with $S = C \cap D$. Then there exists a joint distribution $p \in \Delta(2^{\mathbf{V}})$ with $p \upharpoonright C = q_C$ for every $C \in \mathcal{C}$, and p factorizes over the cliques of G^*. Such p is unique if every separator marginal is strictly positive, but* not *in general.*

Proof. Induction on $|\mathcal{C}|$. If $|\mathcal{C}| = 1$ take $p = q_C$.

Let $|\mathcal{C}| \geq 2$ and pick a leaf cluster C of T with neighbour D and separator $S = C \cap D$. Let $T' = T - C$; by the running-intersection property $C \setminus S$ meets no cluster of T', and $(q_{C'})_{C' \in T'}$ is still separator-consistent, so the inductive hypothesis gives a joint p' over $\mathbf{V}' = \bigcup_{C' \in T'} C'$ with $p' \upharpoonright C' = q_{C'}$ for all $C' \in T'$. Note $p' \upharpoonright S = (p' \upharpoonright D) \upharpoonright S = q_D \upharpoonright S = q_C \upharpoonright S =: q_S$, using separator consistency.

Define a conditional kernel on the fresh atoms $C \setminus S$: for each assignment s of S,

$$\kappa(\cdot \mid s) = \begin{cases} q_C(\cdot\,, s)/q_S(s), & q_S(s) > 0, \\ \delta_{c_0}, & q_S(s) = 0, \end{cases}$$

where c_0 is any fixed assignment of $C \setminus S$. This is well defined: when $q_S(s) > 0$ the right-hand side is non-negative and sums over $C \setminus S$ to $q_S(s)/q_S(s) = 1$, so it is a distribution. Now set $p(x) = p'(x_{\mathbf{V}'}) \cdot \kappa(x_{C \setminus S} \mid x_S)$.

p is a distribution: it is non-negative, and summing over $x_{C \setminus S}$ first gives p', which sums to 1.

The arbitrary choice is irrelevant. On $\{x : q_S(x_S) = 0\}$ we have $p'(x_{\mathbf{V}'}) = 0$, because $p' \upharpoonright S = q_S$ assigns zero mass to that event and $p' \geq 0$. Hence p vanishes there regardless of κ, so p does not depend on c_0. (This is also the precise sense in which the familiar quotient formula $p = \prod_C q_C / \prod_{(C,D)} q_S$ is legitimate: a zero separator cell forces every contributing cluster cell to zero, since $q_S = q_C \upharpoonright S$ is a sum of non-negative q_C entries, so numerator and denominator vanish together and the term is assigned 0.)

Cluster marginals. For $C' \in T'$: summing p over $x_{C \setminus S}$ gives p', so $p \upharpoonright C' = p' \upharpoonright C' = q_{C'}$. For C itself: $p \upharpoonright C(c, s) = \big(\sum \text{ of } p \text{ over } \mathbf{V}' \setminus S \text{ at fixed } x_S = s\big) \cdot \kappa(c \mid s) = p' \upharpoonright S(s)\, \kappa(c \mid s) = q_S(s) \kappa(c \mid s)$, which equals $q_C(c, s)$ when $q_S(s) > 0$ and equals $0 = q_C(c, s)$ when $q_S(s) = 0$. So $p \upharpoonright C = q_C$.

Factorization. Unwinding the induction, p is a product of one non-negative factor per cluster (namely q_C or the kernel κ derived from it, each a function of the atoms of that cluster only), hence factorizes over the cliques of G^*.

Non-uniqueness. If some separator cell $q_S(s) = 0$, then $\kappa(\cdot \mid s)$ was chosen arbitrarily; any other choice yields a joint with the same cluster marginals, since all such joints vanish on that event. Hence p is not unique. When all separator marginals are positive the kernels are forced and the induction determines p uniquely. □

Theorem 4 (CredalJT exactness)**.** *Suppose* (H1) *T has the running-intersection property (true by construction);* (H2) *every LCN sentence is hosted;* (H3) *every LMC assertion is either hosted or separator-entailed in G^*;* (H4) *the query atom and all evidence atoms lie together in some cluster. Then for every such atom a,*

$$\min_{q\in\mathcal{G}} \rho_a(q) = \min_{p\in\mathcal{D}_{\mathrm{LCN}}} \rho_a(\pi(p)) = \underline{P}(a), \qquad \max_{q\in\mathcal{G}} \rho_a(q) = \max_{p\in\mathcal{D}_{\mathrm{LCN}}} \rho_a(\pi(p)) = \overline{P}(a),$$

and likewise for a conditional query $P(a \mid e)$ subject to Remark 14. Moreover the projection *half holds without* (H2)/(H3)*, so `CredalJT` is always a valid* outer *bound.*

Proof. First note the read-off identity: for any joint p, $\rho_a(\pi(p)) = P_p(a)$ (resp. $P_p(a \mid e)$), because ρ_a sums the cells of $\pi(p)_{C_a} = p \restriction C_a$ in which a (and e) hold, which by definition of a marginal equals $P_p(a)$ (resp. the ratio). So both sides optimize the *same* functional.

Projection $\mathcal{D}_{\mathrm{LCN}} \to \mathcal{G}$ (needs only H1, H4). Let $p \in \mathcal{D}_{\mathrm{LCN}}$ and set $q_C := p \restriction C$, i.e. $q = \pi(p)$. Each q_C is a distribution (a marginal of one). Each separator equality holds because both $q_C \restriction S$ and $q_D \restriction S$ equal $p \restriction S$. Each hosted constraint c holds: by Lemma 1 its value depends on p only through $p \restriction \mathrm{scope}(c)$, and since $\mathrm{scope}(c) \subseteq C_h$ we have $q_{C_h} \restriction \mathrm{scope}(c) = p \restriction \mathrm{scope}(c)$, so c holds on the host exactly because it holds for p. Hence $q \in \mathcal{G}$, with $\rho_a(q) = P_p(a)$. Therefore every value achievable over $\mathcal{D}_{\mathrm{LCN}}$ is achievable over $\mathcal{G}$, giving $\min_{\mathcal{G}} \rho_a \le \min_{\mathcal{D}_{\mathrm{LCN}}}$ and $\max_{\mathcal{G}} \rho_a \ge \max_{\mathcal{D}_{\mathrm{LCN}}}$: the cluster interval *contains* the true interval. This is the outer-bound claim, and it used neither H2 nor H3.

Reconstruction $\mathcal{G} \to \mathcal{D}_{\mathrm{LCN}}$ (needs H2, H3). Let $q \in \mathcal{G}$. By H1 and Lemma 6 there is a joint p with $p \restriction C = q_C$ for every cluster, factorizing over the cliques of G^*. We check $p \in \mathcal{D}_{\mathrm{LCN}}$.

Sub-scope consistency: if $A \subseteq C$ for some cluster C, then $p \restriction A = (p \restriction C) \restriction A = q_C \restriction A$, since marginalizing in stages agrees with marginalizing at once.

Hosted constraints (H2 and the hosted part of H3): let c be hosted on C_h. By the previous paragraph $p \restriction \mathrm{scope}(c) = q_{C_h} \restriction \mathrm{scope}(c)$, and q satisfies c on C_h by definition of $\mathcal{G}$; by Lemma 1 c therefore holds for p.

Separator-entailed LMC (the other part of H3): let $(X \perp\!\!\!\perp Y \mid S)$ be separator-entailed, so S separates X from Y in G^*. Since p factorizes over the cliques of G^*, it satisfies the *global Markov property* of G^*: factorization implies global Markov for any undirected graph, and crucially this direction requires *no* positivity assumption (Lauritzen, *Graphical Models*, Prop. 3.8; only the converse, global Markov $\Rightarrow$ factorization, needs $p > 0$ via Hammersley–Clifford). Global Markov states that if S separates A from B in G^* then $A \perp\!\!\!\perp B \mid S$ under p; applying it to the connected components separated by S and then restricting to the subsets X and Y of those components (conditional independence is preserved under taking subsets, by the decomposition property of conditional independence) gives $X \perp\!\!\!\perp Y \mid S$ under p, i.e. (2) holds. Note this assertion was never imposed on $\mathcal{G}$: it holds automatically for the reconstructed p.

Hence $p \in \mathcal{D}_{\mathrm{LCN}}$, and $\rho_a(q) = \rho_a(\pi(p)) = P_p(a)$ because $\pi(p) = q$ on the host cluster. So every value achievable over $\mathcal{G}$ is achievable over $\mathcal{D}_{\mathrm{LCN}}$, giving the reverse inequalities. Combined with the projection half, the optima coincide. For a conditional query, by H4 the read-off is a functional of the single cluster C_a and both maps preserve q_{C_a}, so the same argument applies verbatim. □

Remark 13 (Reading the proof)**.** (a) The argument never mentions chains: exactness is a property of the *junction tree*, via RIP, hosting, separator-entailment and locality. (b) The projection half is unconditional, so `CredalJT` is *never unsound*: its interval always contains the truth, and H2/H3 decide only tightness. (c) H3 is *sufficient, not necessary*: on `examples/asia.lcn` and `examples/polytree.lcn` some assertions are neither hosted nor separator-entailed, yet `CredalJT` is empirically exact — the unentailed assertions simply do not bind. Where H3 fails *and* binds,

the result is a valid but loose outer bound: on `examples/smokers.lcn` the six large-scope LMCs of the undirected component $F_1F_2F_3$ are unhostable and unentailed (within the width-6 cluster $\{F_1, F_2, F_3, S_1, S_2, S_3\}$ no separator divides S_3 from F_1), and `CredalJT` returns $P(F_1) = [0, 1]$.

Remark 14 (The denominator floor is a soundness hypothesis)**.** For conditional queries the implementation linearizes the ratio with an auxiliary variable and, under SCIP, adds a denominator floor $\mathbf{1}_e^\top q_{C_a} \geq \varepsilon$ with `_DEN_FLOOR`$= 10^{-6}$ (`junction_nlp.py:74,662`). This is an extra constraint on $\mathcal{G}$ that $\mathcal{D}_{\text{LCN}}$ does not have, so the projection half of Theorem 4 — and with it the outer-bound guarantee — holds only for joints with $P(e) \geq \varepsilon$. If $\min_{p \in \mathcal{D}_{\text{LCN}}} P(e) < \varepsilon$ the returned conditional interval can be *unsound*. This is a soundness caveat, not merely a tightness one, and it is the exact analogue of Credal VE's `if total <= 0: continue` (Remark 7).

Corollary 2 (Exactness on `ktree-fr` at treewidth cost)**.** *On every* `ktree-fr` *instance, H1–H4 hold, so* `CredalJT` *returns the exact* $\mathcal{D}_{\text{LCN}}$ *marginal bounds at cost* $O(n\, 2^{k+1})$.

Proof. A k-tree is chordal, and the generator renders it as a DAG in construction order in which every atom conditions on the *full conjunction* of its k clique-parents. Those parents are pairwise adjacent in the k-tree, so moralization adds no edge and triangulation is a no-op: G^* is the k-tree itself and its maximal cliques — the clusters — are exactly the $(k{+}1)$-cliques, giving H1 by construction. Every sentence is either a root marginal or scoped to a child with its $\leq k$ parents, which co-occur in a cluster: H2. Each LMC assertion produced by the Local Markov Condition has as its conditioning set a full clique-separator of the k-tree, which separates the two sides in G^*, so each is hosted or separator-entailed: H3. Any single query atom lies in a $(k{+}1)$-clique: H4. Theorem 4 applies, and the cost is $\sum_C 2^{|C|}$ over $O(n)$ clusters each of size $k{+}1$. □

Remark 15 (What the `ktree-fr` corollary does and does not claim)**.** Corollary 2 is a statement about *cost*: exactness at $O(n2^{k+1})$ rather than 2^n. It should *not* be read as "the one loopy topology where a treewidth-bounded engine beats the strong-extension engines on accuracy". Example 5 shows Credal VE is also exact on a loopy realizable k-tree; on the larger $k{=}3$ instance of Example 2 Credal VE fails not by being loose but by not finishing (Remark 6). The separation between `CredalJT` and Credal VE on this family is one of tractability.

Example 9 (Verified end to end)**.** Example 2 is the worked case: on `ktree_fr_k3_n5_1.lcn` the two maximal cliques are $\{x_0, x_1, x_3, x_4\}$ and $\{x_1, x_2, x_3, x_4\}$ (treewidth 3), the sole LMC assertion $(x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3, x_4)$ has conditioning set equal to the separator between them, and `CredalJT` reproduces the certified `ExactInference(global)` bounds on all five atoms. Further checks: on `examples/chain.lcn` the clusters are the 7 edge-pairs (treewidth 1: 28 NLP variables against a $2^8 = 256$ joint); on `examples/polytree.lcn` the collider cliques $\{x_0, x_1, x_2\}$ and $\{x_3, x_4, x_5\}$ give treewidth 2; and on the loopy Example 5 `CredalJT` matches the certified oracle on every atom.

6 ARIEL

ARIEL (`lcn/inference/marginal/ariel.py`) is included here for reference: it targets $\mathcal{D}_{\text{LCN}}$ directly on the primal-graph factor graph, without compiling a credal network, so it sits outside the $\mathcal{D}_{\text{SE}}$ family.

6.1 The algorithm

A *factor node* groups all sentences sharing the *same* atom scope and is joined to each variable node in that scope (`utils/factor_graph.py`). Two message kinds are passed to a fixed point:

- *Variable-to-factor* $v \to f$: the intersection of the other incoming intervals, $\underline{\ell}_{v\to f} = \max_{f'\neq f} \underline{\ell}_{f'\to v}$ and $\overline{u}_{v\to f} = \min_{f'\neq f} \overline{u}_{f'\to v}$.
- *Factor-to-variable* $f \to n$: minimize and maximize $P(n)$ over a *local* NLP on the factor's own joint $\vec{p} \in \Delta(2^{|\text{scope}(f)|})$, subject to (i) f's sentences; (ii) the LMC equalities (2) whose *entire* scope fits inside scope(f); and (iii) for each other boundary variable m, the box constraint $\underline{\ell}_{m\to f} \le P(m) \le \overline{u}_{m\to f}$.

The answer for atom v is the intersection $\underline{P}(v) = \max_f \underline{\ell}_{f\to v}$, $\overline{P}(v) = \min_f \overline{u}_{f\to v}$.

Lemma 7 (Message soundness)**.** *For every $P^\star \in \mathcal{D}_{\text{LCN}}$, every edge and every iteration t, $P^\star$'s marginal of the message's variable lies in the message interval. Consequently ARIEL's returned interval is a valid* outer *bound w.r.t. $\mathcal{D}_{\text{LCN}}$ — on* any *topology, loopy included.*

Proof. Induction on t, simultaneously over all edges. At $t = 0$ every message is $[0, 1]$.

Assume the claim at t. For a *variable-to-factor* message, the update is an intersection of intervals each of which contains $P^\star(v)$ by the inductive hypothesis, hence so does the intersection.

For a *factor-to-variable* message $f \to n$, consider the point $\vec{p}^\star := P^\star \restriction \text{scope}(f)$. We claim it is feasible for f's local NLP. It lies in the simplex, being a marginal of a distribution. It satisfies f's sentences: each such sentence has atoms(s) $\subseteq$ scope(f) and holds for $P^\star$ (as $P^\star \in \mathcal{D}_{\text{LCN}}$), so by Lemma 1 it holds for $\vec{p}^\star$. It satisfies the in-scope LMC equalities by the same locality argument. And for each other boundary variable m, the box constraint $\underline{\ell}_{m\to f} \le P(m) \le \overline{u}_{m\to f}$ is satisfied because $\vec{p}^\star$'s marginal of m is $P^\star(m)$, which lies in the incoming interval by the inductive hypothesis. Hence $\vec{p}^\star$ is feasible, so the minimum of $P(n)$ over the local NLP is $\le P^\star(n)$ and the maximum is $\ge P^\star(n)$: the new message contains $P^\star(n)$.

The final read-off intersects intervals all containing $P^\star(v)$, so it contains it too. Since $P^\star \in \mathcal{D}_{\text{LCN}}$ was arbitrary, the returned interval contains $\{P(v) : P \in \mathcal{D}_{\text{LCN}}\}$, which is the outer-bound claim. No topological hypothesis was used. □

Remark 16 (The implicit independencies are a relaxation, not an added constraint)**.** It is sometimes said that ARIEL "asserts" pairwise independencies among a factor's boundary variables that are absent from the LMC — for the polytree collider factor $\{x_0, x_1, x_2\}$, for instance, the false $(x_0 \perp\!\!\!\perp x_2)$ and $(x_1 \perp\!\!\!\perp x_2)$. This would be alarming if those assertions were *imposed*, since adding independencies shrinks the feasible set and could produce an unsound inner bound. They are not imposed. The routine that materializes them (`_message_independencies`, `ariel.py:652`) is reached only from `analyze()` (line 770) and never feeds a solver; the local NLP contains only the simplex, the factor's sentences, the in-scope LMC equalities and the neighbour boxes, as listed above. The implicit independencies describe what a marginal-interval message *fails to convey*, i.e. an information loss that *enlarges* the effective feasible set. They therefore threaten tightness, never validity — consistent with Lemma 7, which shows the true $P^\star$ is always feasible.

Proposition 6 (Tightness on a tree of small factors)**.** *Suppose (i) the factor graph is a tree; (ii) every factor scope has size ≤ 2, so each factor-to-variable update has at most one other boundary variable; and (iii) the LCN is fully realizable (Definition 2), and each local NLP is solved to global optimality. Then the converged intervals equal the exact $\mathcal{D}_{\text{LCN}}$ bounds.*

Proof. By Lemma 7 the intervals contain the truth, so it suffices to show each endpoint is *attained* by some $P \in \mathcal{D}_{\text{LCN}}$. Root the factor tree at the queried atom and induct on subtree depth.

For a leaf factor f with scope $\{n\}$ or $\{m, n\}$ where m has no other factor: the local NLP's constraints are exactly f's sentences (plus a vacuous box), so an optimizer $\vec{p}$ is a distribution on

scope(f) satisfying them, and by (iii) it extends to a member of $\mathcal{D}_{\mathrm{LCN}}$ — this is where realizability is used: each sentence constrains one family's conditional alone, so the extension may set the remaining families' conditionals freely without disturbing any sentence, and by Proposition 2 the resulting product lies in $\mathcal{D}_{\mathrm{LCN}}$. Hence the endpoint is attained.

For an internal factor f with scope $\{m, n\}$: by (ii) there is exactly one other boundary variable m, and by (i) removing m disconnects f's side of the tree from the rest, so the atoms beyond m form a separate subtree. By induction the incoming interval endpoints on m are attained by distributions on the far side. Because m is binary, its marginal is the single scalar $t = P(m{=}1)$, and the incoming interval is $[\underline{t}, \overline{t}]$; by induction every value in $\{\underline{t}, \overline{t}\}$ is realized by a far-side member of $\mathcal{D}_{\mathrm{LCN}}$, and by continuity (the far-side constraint set is connected) so is every t in between. The local NLP optimizes $P(n)$ over distributions on $\{m, n\}$ with $P(m) = t$ for some admissible t and with f's sentences satisfied. Its optimizer selects a value $t^\star$ and a conditional $P(n \mid m)$; by realizability that conditional is constrained only by f's own sentences, and by the induction $t^\star$ is realized on the far side — independently, since the two sides share only m. Composing the two gives a member of $\mathcal{D}_{\mathrm{LCN}}$ attaining the endpoint. □

Remark 17 (The general claim, and why we state it as cited)**.** ARIEL is asserted to be exact on *all* singly-connected LCNs — Theorem 1 of Marinescu et al., *Approximate Inference in Logical Credal Networks* (IJCAI-2023) — and the shipped engine does reproduce the exact bounds on `chain.lcn`, `tree.lcn` and `polytree.lcn` to four decimals. We state that result with attribution rather than reproving it, because Proposition 6's hypothesis (ii) is not merely a convenience of the proof. At a collider factor with scope $\{m_1, m_2, n\}$ — which occurs on a *polytree*, hence on a singly-connected graph — the local NLP constrains $P(m_1)$ and $P(m_2)$ *separately* and then ranges over *all* joint distributions of (m_1, m_2) with those marginals, including correlated ones. Two binary variables with fixed marginals admit a one-parameter family of joints, of which the LMC's co-parent independence ($m_1 \perp\!\!\!\perp m_2$) picks out a single point. So the relaxation is genuine and does not vanish for binary atoms: single-connectedness alone does not obviously supply tightness, and we do not claim a proof that it does. What is certain, and proved above, is Lemma 7: ARIEL is always a valid outer bound. This is also the entry the summary tables need.

Remark 18 (Solver and slack caveats)**.** Two implementation facts qualify Lemma 7 in practice. The local NLPs are nonconvex (cleared Type-2 rows and bilinear LMC equalities) and are solved by ipopt, a *local* method: a local optimum that falls short of the true min/max yields a too-narrow message, which propagates and can render the result unsound — the same hazard as `ExactInference`-local (Remark 20). Additionally the neighbour box constraints are enforced with slack variables under a finite penalty, so they can be violated when the penalty is not binding. Both are reasons to treat ARIEL's output as an approximation rather than a certificate.

Example 10 (The loopy boundary, with an uncoupled control)**.** Proposition 6's hypothesis (i) is necessary. Take the diamond $A \to \{B, C\}$, $\{B, C\} \to D$:

$$0.4 \le P(A) \le 0.6; \quad P(B \mid A), P(C \mid A) \text{ transitions}; \quad P(D \mid B, C) \text{ (four rows)}; \quad \boxed{P(B \wedge C) \le 0.25}.$$

The boxed cross-family sentence creates the factor $\{B, C\}$ and a second cycle, so the factor graph is loopy. Measured:

atom	ARIEL	`CredalJT` (exact)	verdict
A	$[0.4000, 0.6000]$	$[0.4000, 0.6000]$	exact
B	$[0.3600, \mathbf{0.6400}]$	$[0.3600, \mathbf{0.5700}]$	*outer* (+0.07)
C	$[0.3600, \mathbf{0.6400}]$	$[0.3600, \mathbf{0.5700}]$	*outer* (+0.07)
D	$[0.3160, \mathbf{0.6840}]$	$[0.3160, \mathbf{0.5940}]$	*outer* (+0.09)

The factor $\{B, C\}$ *knows* the cap but can only *report* marginal intervals on B and C; the joint restriction is lost at the message boundary, so the cap never tightens the rest. *Control:* removing the cap makes ARIEL and `CredalJT` agree on all four atoms ($A = [0.4, 0.6]$, $B = C = [0.36, 0.64]$, $D = [0.316, 0.684]$), which localizes the looseness to the cross-family coupling rather than to the loop alone. Consistently with Lemma 7, ARIEL is outer, never inner, throughout.

Remark 19 (Shipped status: the factor-grouping bug is fixed)**.** An earlier version of `FactorGraph._make_factor_nodes` created the first factor before the grouping loop and then reused the label `f1` for the next new-scope factor (the index was incremented after use), silently overwriting — and dropping the sentences of — the first factor. Since sentences are sorted by decreasing scope, the casualty was always the largest-scope group, so it returned vacuous $[0, 1]$ interior marginals on chain/tree/polytree. The fix (each new factor gets a fresh post-incremented label) is present at `utils/factor_graph.py:225--237`. This is worth recording because the companion note `ariel_exactness.tex` documents the post-fix behaviour without mentioning the bug.

7 The exact oracle, and its limits

`ExactInference` (`marginal/exact.py`) solves, per atom, a min and a max of $P(a)$ (or the ratio $P(a \mid e)$) over the full 2^n joint subject to *all* sentences and *all* LMC equalities — i.e. directly over $\mathcal{D}_{\mathrm{LCN}}$ of (3). It is the definitional target, not a member of the credal-network family.

Proposition 7 (Certified global; local unreliable)**.** *With `solver="global"` (SCIP spatial branch-and-bound) and a reported gap $\leq$ `gap_tol` with status `confirmed`, `ExactInference` returns the exact $\mathcal{D}_{\mathrm{LCN}}$ bounds. With `solver="local"` (ipopt with an SLSQP fallback) the returned interval is* unreliable *in the sense of Definition 1: it may be spuriously inner.*

Proof. The optimization model is by construction (3), whose feasible set is exactly $\mathcal{D}_{\mathrm{LCN}}$; hence any *globally* optimal value equals the corresponding true bound, and SCIP's spatial branch-and-bound certifies global optimality up to the reported gap — when the gap is 0 and the status is `confirmed`, the value is the exact bound. For the local backend, the feasible region is nonconvex (the LMC equalities (2) and the cleared Type-2 rows (1) are bilinear), so a local NLP method can converge to a stationary point whose objective is strictly interior to the true extremum; the multi-restart and SLSQP fallback mitigate but do not certify this, so no containment in either direction is guaranteed. □

Remark 20 (`ExactInference`-local over-tightens)**.** The failure in Proposition 7 is observed, not hypothetical: on `examples/alarm.lcn` the brute-force and SCIP cross-checks (`verify_bruteforce.py`, `verify_scip.py`) put $P(A) = [0.087, 0.100]$ and $P(E) = [0.050, 0.074]$, whereas `solver="local"` reports the spuriously tighter $P(A) = [0.092, 0.100]$, $P(E) = [0.050, 0.073]$ — an inner, unsound interval. Never use the local backend as a guarantee.

Remark 21 (Certified-global does not always terminate)**.** The global backend is not a universal oracle either, and its per-atom `status` field must be read. On the five-atom `alarm.lcn` a run with a 45 s per-solve limit returned $A = [0, 0.1]$, $B = [0.1, 1.0]$, $E = [0.05, 1.0]$, $C = D = [0, 1]$ — but the status shows why: the lower solve for A and the *upper* solves for B and E came back `unsolved`, so those endpoints are fallbacks rather than bounds, while C and D are `confirmed` genuinely vacuous. These wide values are consistent with, not a refutation of, the certified $[0.087, 0.100]$ etc. of Remark 20; they are simply uncertified. Practical consequences: (i) treat an `ExactInference`-global number as exact only when its status is `confirmed`; (ii) on instances like this one `CredalJT`

— which solved every example in this document in seconds — is the more trustworthy engine, despite n being small enough that the 2^n model is formulable.

8 Summary tables

Table 2: Bound character per engine. "Targets" is the feasible set the engine optimizes; "character" is per Definition 1 *against that set*. The last column gives the condition for the returned interval to equal the $\mathcal{D}_{\text{LCN}}$ truth. Throughout, `method="linear"` is assumed (Remark 3).

Engine	Targets	Character (vs. target)	Exact w.r.t. $\mathcal{D}_{\text{LCN}}$ when	Cost
`ExactInference` (global)	$\mathcal{D}_{\text{LCN}}$	exact if `confirmed`	status `confirmed`	2^n, SCIP
`ExactInference` (local)	$\mathcal{D}_{\text{LCN}}$	unreliable	— (may over-tighten)	2^n, ipopt
`CredalJT` (D5)	$\mathcal{D}_{\text{LCN}}$	outer always; exact if H1–H4	H1–H4 (Thm. 4)	$O(n2^{w+1})$
Credal VE	$\mathcal{D}_{\text{SE}}$	exact (uncond., Thm. 1)	Prop. 2 holds	per-target elim.
Credal CTE (ϵ=0)	$\mathcal{D}_{\text{SE}}$	**inner** (Prop. 3)	clean s.c.; no cross-family sent.	two-pass tree
ApproxLP	$\mathcal{D}_{\text{SE}}$	*inner* (Thm. 3)	Prop. 2 + no stall	coord. descent
Interval BP (`interval`)	$\mathcal{D}_{\text{SE}}$	outer; exact on tree (Thm. 2)	tree + Prop. 2	loopy messages
Interval BP (`variational`)	$\mathcal{D}_{\text{SE}}$	*inner*	—	mean field
Credal IJGP	$\mathcal{D}_{\text{SE}}$	outer; exact if $i \geq w$	$i \geq w$ + Prop. 2	2^i/cluster
ARIEL	$\mathcal{D}_{\text{LCN}}$	outer always (Lem. 7)	Prop. 6; cited Thm. 1	factor messages
SCCP / SCCP-ARIEL	$\mathcal{D}_{\text{LCN}}$	outer	trivial SCCs	per-SCC NLP

Table 3: Bound character by topology, for unconditional marginals. "ex" = exact w.r.t. $\mathcal{D}_{\text{LCN}}$, "out" = valid outer bound, "in" = inner (unsound as a guarantee), "n/f" = did not finish. Entries marked † were measured for this document.

Topology	Exact-glob	CredalJT	CVE	CTE	ApproxLP	IBP-int
Markov chain, tree, polytree (realizable)	ex	ex	ex	ex	ex	ex
Chain graph with blocks (`alarm`)	ex*	ex	ex	ex	ex	ex
Cross-family sentence (`d4_biting`)	ex†	ex†	ex†	**in**†	ex†	ex†
Non-realizable Type-1 (Ex. 4)	ex†	ex†	out†	out	out†	out†
Non-realizable Type-2 (Ex. 3)	ex†	ex†	out†	out	out	out
Loopy k-tree, realizable, k=2 (Ex. 5)	ex†	ex†	**ex**†	—	—	—
Loopy k-tree, realizable, k=3 (Ex. 2)	ex†	ex†	**n/f**†	—	—	—
Undirected block (`smokers`)	ex	**out**	out	out	in	out

* On `alarm` the global backend leaves several endpoints `unsolved` (Remark 21), and the *local* backend over-tightens (Remark 20); the certified values come from the verifiers.

Remark 22 (What changed relative to earlier versions of this table)**.** Three rows were corrected on the basis of measurement. (a) Credal CTE is no longer listed as "exact over $\mathcal{D}_{\text{SE}}$": it is *inner* once a cross-family sentence appears (Proposition 3). (b) The loopy k-tree rows previously read "out" for Credal VE on the grounds that the class is loopy; at k=2 Credal VE is measurably *exact*, and at k=3 its problem is non-termination, not looseness (Example 5, Remark 6). (c) ApproxLP's character is now stated against $\mathcal{D}_{\text{SE}}$, since w.r.t. $\mathcal{D}_{\text{LCN}}$ it can be outer (Example 7).

9 Decision guide

- **Need a guarantee, small n:** `ExactInference` with `solver="global"` — and check the per-atom `status`, since `unsolved` endpoints are not bounds (Remark 21). Never rely on `solver="local"`.
- **Need exactness at scale with bounded treewidth:** `CredalJT` (D5). Exact under H1–H4, always a valid outer bound otherwise, and in practice the most trustworthy engine here — including on instances where the 2^n oracle fails to certify. Mind Remark 14 for conditional queries.
- **All singleton marginals over $\mathcal{D}_{\mathrm{SE}}$, exactly:** Credal VE, for unconditional queries (Theorem 1). Prefer it over Credal CTE whenever a cross-family sentence is present.
- **A two-sided bracket:** pair ApproxLP (inner over $\mathcal{D}_{\mathrm{SE}}$) with Credal VE (exact over $\mathcal{D}_{\mathrm{SE}}$) on the same vertices; coinciding endpoints certify the $\mathcal{D}_{\mathrm{SE}}$ bound and differing ones localize a coordinate-descent stall (Corollary 1).
- **Fast and singly connected:** Interval BP `method="interval"` or ARIEL. Do not use Interval BP for conditional queries with evidence on descendants (Remark 10).
- **Suspect a loose bound?** Check realizability first (Definition 2): a single non-realizable sentence is the most common cause, and it is a syntactic property you can check by inspection via Lemma 2. Loopiness alone is *not* a reason to expect looseness (Example 5).
- **Tightening:** D1–D3 are free build-time knobs for every $\mathcal{D}_{\mathrm{SE}}$ engine, but note that D1 breaks the convexity the vertex machinery assumes (Remark 3); D4/D5 address Gap B.

Provenance of the numbers

Every interval reported in Examples 2, 3, 4, 5, 6, 7, 8, 10, in Proposition 3, and in Remarks 10 and 21 was produced by running the shipped engines on the stated instance. Values attributed to `ExactInference`(`global`) are reported as exact only where the per-atom status is `confirmed` with gap 0.

Two figures are *not* re-derived here and are cited from the verifiers instead: the certified `alarm` marginals of Remark 20 (`verify_bruteforce.py` and `verify_scip.py`), and the pre-fix Interval BP numbers of Remark 11. The coupled diamond of Example 10 is not shipped as a `.lcn` file; it was reconstructed from the description in `ariel_exactness.tex` and reproduces that note's numbers.

Companion notes

Per-engine detail lives in `cve_exactness.tex`, `ccte_exactness.tex`, `ibp_exactness.tex`, `approxlp_exactness.tex`, `ariel_exactness.tex`, `cjt_exactness.tex`, `strong_extension_exactness.tex` and `tighter_approximation.tex` (the D1–D5 taxonomy). Where this document and a companion note disagree, the discrepancies are deliberate and are flagged at Remarks 9, 5, 15 and 17, and in Lemma 6's non-uniqueness clause.